A Programmatic Document Analysis Environment for Patent Examination

Shanaya Malik
shanaya_malik@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

Kelvin Zhao
kelvinyz@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

ABSTRACT

CCS CONCEPTS

• **Human-centered computing** → **Empirical studies in HCI**; *HCI design and evaluation methods*; • **Software and its engineering** → *Domain-specific languages.*

KEYWORDS

patent examination, prior art search, domain-specific language, document analysis, contextual inquiry, sensemaking, information work, HCI

ACM Reference Format:
Shanaya Malik and Kelvin Zhao. 2026. A Programmatic Document Analysis Environment for Patent Examination. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 MOTIVATION

The stakes of patent examination. Patent examination is a high-stakes, knowledge-intensive activity in which trained examiners must locate prior art—existing patents, publications, or other disclosures—that *anticipates* or *obviates* each claim in a pending patent application. Because issued patents carry legal force, the quality of examination directly affects the validity of the intellectual property record and the downstream costs of litigation and re-examination. Yet the core task of finding prior art remains largely manual: an examiner reads the claims, identifies the key technical concepts, formulates search queries, retrieves candidate documents, reads those documents to find relevant passages, and then records exactly *where* in each document the evidence appears so it can be cited in an office action. For complex technical art units—software, biomedical devices, quantum computing—this cycle can span dozens of documents and hundreds of individual claim elements per application.

Why existing tools are insufficient. Current search infrastructure, including the USPTO's internal EAST and WEST systems as well as commercial tools such as PatSnap and Derwent Innovation, treats this workflow at the document level. A query returns a ranked list of patent documents; the examiner then opens each document and manually locates the relevant section. Tools surface keyword hit *counts*, but do not distinguish whether a hit appears in the abstract, the background, the detailed specification, or the claims—sections that carry very different evidentiary weight. Passage-level retrieval, section-aware filtering, and metadata-constrained queries are not available as first-class capabilities in any tool accessible to working examiners.

Evidence from practitioners. Through a contextual inquiry study with ten practicing patent examiners (reported in full in Sections 3–4), we identified five recurring needs that current tooling does not address. Two of these needs are particularly relevant to the design of a document analysis environment. First, examiners need *section-aware retrieval*: P05 explicitly described wanting hits in the background and detailed description sections, not in the abstract or claims, because the specification is where the technical teaching that maps to claim elements is found. Section type is a structural property of every patent document, yet no current tool exposes it as a query parameter. Second, examiners need *pinpoint citation*: once a relevant passage is located, examiners must record its position in the document for citation in the office action. For post-2001 USPTO publications, this position is a numbered paragraph anchor (e.g., `[0042]`); for pre-AIA granted patents, it is a column-and-line number. This pinpoint address is embedded in the document's structure, yet retrieving by it is not possible in any current tool.

The root cause: documents as opaque blobs. A further structural problem unifies all five needs: patent documents are treated as opaque text blobs. Queries operate over full-text indexes; results are document-level rankings. The rich internal structure of a patent—its sections, numbered paragraphs, claim hierarchy, and embedded metadata—is invisible to the query layer. An examiner who wants to find all passages in the detailed specification of a particular assignee's patents that discuss a specific technical mechanism must open each document, scroll to the relevant section, and scan manually. This is not a failure of individual tools but a consequence of the retrieval model: when documents are the unit of retrieval, there is no path to passage-level precision.

Our approach and expected outcomes. We propose a different approach. Rather than building another document-level search interface, we expose patent documents as *structured, queryable data.* Our system parses each patent into a `Document` → `Section` → `Passage` hierarchy and extracts all available metadata fields. Examiners then write queries in a minimal domain-specific language (DSL) that combines structural filters—`section:SPECIFICATION`, `paragraph:0042`—with content filters and metadata constraints, all composable with Boolean operators. The query executes over the passage-level representation and returns matched passages

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

with their full provenance: which document, which section, which paragraph anchor, and neighboring context.

Why a DSL fits this population. This design is motivated by two key insights from our need-finding. First, patent examiners are not casual users: they are technical specialists who write Boolean search strings as a core job skill, and several participants described current query interfaces as too limited rather than too complex (P04, P07, P10). A textual DSL is therefore appropriate for this population in a way it might not be for a general consumer audience. Second, the iterative, refinement-based search strategy that examiners actually use (Need 5) is precisely what a composable query language supports: an examiner can start with a broad `section:CLAIMS` filter, observe the results, narrow with `AND contains:"neural interface"`, and further restrict with `AND NOT paragraph:0001` to skip boilerplate preambles. Each refinement is a surgical edit to the query string, not a round-trip through a separate GUI tool.

The contributions of this paper are as follows:

- We describe a passage-level document model for patent text that exposes section type, numbered paragraph anchors, claim structure, and rich patent metadata as first-class query targets, grounded in the structural needs identified through contextual inquiry.
- We define a minimal DSL for composable patent queries, supporting `section:`, `contains:`, `paragraph:`, `cpc:`, and `meta.` filters with full Boolean logic (`AND`, `OR`, `NOT`, parentheses), and describe its design rationale in terms of the five needs from our study.
- We present a prototype implementation—a FastAPI backend with a React frontend—that parses USPTO-format patent documents (both plain-text and text-extractable PDF) into the passage-level model and executes DSL queries with passage-level results, including neighboring context and match provenance.
- We characterize the open technical challenges in this design space, including document parsing fidelity, query expressiveness limits, and the performance tradeoffs of in-memory passage indexing at corpus scale.

2 RELATED WORK

2.1 Sensemaking and Information Foraging in Professional Technical Work

2.2 Tools for Patent Search and Document Analysis

2.3 Domain-Specific Languages for Structured Document Access

2.4 Contextual Inquiry for Studying Specialized Workflows

3 USER STUDY DESIGN AND PROCEDURE

The study was designed to address the research question: *What kinds of problems do technical patent examiners face during the iterative process of discovering, filtering, and synthesizing prior art to evaluate the novelty of complex technical claims?* This question was scoped to examiners in technical art units (such as software, biomedical engineering, or quantum computing), where the demands of translating between legal claim language and technical implementation are particularly demanding. We chose not to pre-specify hypotheses, since the study was exploratory, need-finding.

The participants were recruited through purposive and snowball sampling, beginning with two professional contacts who shared a recruitment message with colleagues and provided us with additional contacts. To supplement the remaining number of participants, we used cold messaging on LinkedIn to target individuals whose profiles indicated professional experience in technical patent examination. More specifically, the inclusion criteria required participants to be at least 18 years old, fluent in English, and currently or formerly employed in patent examination, with experience evaluating complex technical claims. We excluded individuals who exclusively examine non-technical patents and anyone unable to demonstrate their workflow using only publicly available documents.

Each session lasted approximately 45 minutes and was conducted remotely via Zoom. The sessions began with a brief unrecorded introduction of the participant and researcher, followed by an overview of the project scope and agenda for the session. The core of the session (~30–45 minutes, recorded) was a contextual inquiry following the Master-Apprentice model, in which the participant shared their screen and demonstrated their prior-art search and synthesis process for a given public patent while thinking aloud, with clarifying questions asked at natural pause points [8]. This was followed by a synthesis segment focused on how the participant maps and documents the relationship between prior art and specific claim elements, and concluded with a closing discussion addressing the most time-consuming and mentally demanding aspects of the workflow.

Each session produced three data artifacts stored on a password-protected Berkeley bDrive account: (1) a Zoom screen-and-audio recording capturing screen activity, search queries, and think-aloud narration; (2) typed session notes taken in real time; and (3) a post-session debrief document recording emerging patterns. The recordings were transcribed within Zoom itself and reviewed for accuracy. The participants were assigned coded identifiers (P01, P02, etc.), and we consolidated each participant's transcript, session notes, and debrief into a single document as part of our input to our reflexive thematic analysis [11].

4 USER STUDY RESULTS

4.1 Analysis Process

We analyzed the consolidated session documents using reflexive thematic analysis following Braun and Clarke [11]. Each researcher independently conducted an initial round of open coding on their assigned participant's contextual inquiries, applying purely descriptive labels to segments that capture actions, reasoning, breakdowns, and workarounds within each patent examiner's workflow. Since our data include both think-aloud narration and question answers, as well as screen-recorded behavior, our coding explicitly distinguishes between what participants said and what actions they took, allowing us to capture both direct information from explanations and implicit information from instances where difficulties were

not verbally articulated. As a result, many codes described domain-specific interactions, such as refining search queries, tracking claim coverage across references, and using specific internal tools to manage information across documents.

After independent coding, we compared our code sets across assigned participants and iteratively refined them through discussion, focusing first on resolving differences in interpretation and initial grouping, before collaboratively identifying higher-level patterns that showed recurring needs across multiple participants. We treated this stage as an iterative process, merging overlapping codes and preserving any meaningful differences reflecting underlying issues in the participants' workflows. We grouped related codes into candidate themes, tightened their definitions, and verified that each theme was supported by evidence from multiple participants. The five themes we present below represent the needs that emerged most consistently across our participant pool.

4.2 Findings

Need 1: Examiners lack systematic support for generating and managing the synonyms needed to translate legal claim language into effective search queries. A central challenge across sessions was the gap between claim terminology and the terminology used to retrieve relevant prior art; as P04 explained, the applicant "comes up with their own term, so we try to come up with terms that are going to be describing the same thing." P06 described this as a problem of implied language: a term like *modder* in a medical device patent could return results about computer modification, and a term like *wearable* might not appear in a relevant document that uses a synonym, meaning those documents would be filtered out. Examiners relied on a patchwork of ad hoc resources, including a shared synonym crosswalk tool (P04), Google searches (P04), terminology from prior specifications, and personal experience. P06 described a case where an applicant's term for an ophthalmic structure appeared only in their own application, requiring consultation with a senior examiner to identify standard terminology. P07 noted that while examiners are taught about synonyms and Boolean connectors, applying this in unfamiliar art areas requires substantial independent research, alongside occasionally having to consult a senior examiner. Similarly, P10 described getting "stuck with the synonyms" when attempting to expand search queries in unfamiliar or highly technical applications, reinforcing that synonym generation remains an unsupported and cognitively demanding step. Junior examiners were particularly disadvantaged, lacking the tacit vocabulary that senior examiners described as built through years of repetition (P01, P05, P06).

Need 2: Current search tools do not adequately support filtering by filing date, leading to wasted effort and rework. Every participant referenced the effective filing date as a hard constraint on admissible prior art, yet existing tools provided limited support for enforcing it during search. P05 described completing an entire office action based on a reference that turned out to post-date the application's effective filing date, requiring a complete redo. P06 identified a related problem in which search systems frequently confuse publication dates with filing dates, and also confuse non-provisional filing dates with earlier provisional dates that establish actual priority. P06 called this "definitely a very good fix that can be done." P01 noted that the rules for effective filing dates differ between U.S. and foreign applications, making even manual checks non-trivial. P08 described the filing-date constraint as deeply embedded in the workflow, requiring cross-referencing across domestic, provisional, and foreign filings, and P03 had initially demonstrated this in action, showing that of seven initial results, only one had a usable date (but determining this required a manual, time-consuming inspection of each).

Need 3: The transition from search to claim mapping relies on entirely manual, ad hoc methods with little integrated tool support. After identifying promising prior art, examiners must document how specific passages address specific claim elements, which is a process called claim mapping. Every participant performed this using personal, improvised methods outside the search tool. P01 maintained a Word table; P03 copied claims as bullet points and mapped inline; P04 used a personal claim chart; and P05 copied claims into a Word document with parentheses after each element and drew dependency trees by hand. P06 described using pen and paper for claim trees and explained that the in-house software's mapping feature was so inaccurate that "pretty much no one at the USPTO" trusts it—errors in claim dependencies can distort the entire analysis. P07 described the process as lacking elegant tooling, stating that "it's a very manual process, [examiners] just start plopping it in there." P08 described printing claims on paper and annotating them physically to manage cognitive load across simultaneous cases, and P05 explicitly identified this as a design opportunity, stating that integrating claim mapping into the search tool could save a significant amount of time and energy for examiners.

Need 4: Examiners struggle to locate and interpret relevant information within retrieved documents. Examiners reported that tools display keyword hit counts without distinguishing whether hits appear in the specification (the detailed description most useful for mapping), the abstract, or the claims. P05 wanted hits specifically in the background and description sections, not in the abstract or claims, and shared that the ranking system orders results by total hit count regardless of location, forcing examiners to open documents and manually scan for relevant sections. P02 used a similar process of scrolling and combing through CTRL+F results of keywords to find relevant paragraphs. P01 expressed further frustration with granted patents, which require citing by column and line number rather than paragraph number, adding manual overhead when recording where evidence appeared. P09 described that some internal tools support multi-highlighting to track keyword locations across claims, but public tools lack this capability entirely, forcing examiners to flip between separate documents to locate where relevant content appears manually. In practice, P10 also described relying on scanning titles, images, and highlighted keywords to triage relevance, further illustrating that examiners must manually piece together signals across document representations to determine whether a result is worth deeper inspection. Several participants also described needing to read beyond the keyword hit itself to determine whether it was actually meaningful in context, often revisiting multiple sections of the document to interpret how the term was being used. For example, P03 explained that when a result appears potentially relevant, they

cannot rely on keyword matches alone and instead "have to go through the whole patent" to verify whether it actually aligns with the claim, since surface-level indicators like keyword proximity are often insufficient. Together, these patterns suggest that relevance assessment is slowed not only by the difficulty of locating keyword matches but also by the additional cognitive effort required to interpret their significance in complex technical documents.

Need 5: Existing AI-assisted search tools do not align with the iterative, refinement-based search strategy that examiners actually use. Participants with experience using AI or semantic search tools—including the USPTO's similarity search, IP.com, and other platforms—consistently described them as inadequate. P02 emphasized that effective search "really involves appraising the art as you search for it, and refining your search based on that," which current tools do not support. P06 described the similarity search as "hit or miss," identifying the core problem as the tool's inability to distinguish domain context—a medical-device search for "user interface" returns results from banking and unrelated software. P07, while noting IP.com "seems to interpret language a little bit more intuitively," still gravitated toward non-patent sources like technical manuals as a more efficient strategy. P09 echoed the mixed assessment, noting that the similarity search "has mixed results—sometimes I just click that button… and it finds me 300 references, of which 2 or 3 are actually useful. Other times, it's just completely off the mark." P09 also described a workaround of feeding publicly available PG pubs into external AI tools to help devise search strategies, while noting that proprietary restrictions prevent using AI with actual application information. It appeared that, across most participants, AI tools were supplements rather than primary methods, reinforcing that future design should augment iterative workflows rather than replace them.

4.3 Implications for Future Research and Tool Design

Several of our findings align with patterns reported in prior work on professional information work, including the challenges of maintaining context across documents [1], organizing scattered evidence [4], and iterative refinement in expert search [2]. However, our findings also diverge from prior work in important ways. First, while Ko et al. [1] found that developers lose track of reasoning chains across files, patent examiners face an additional structural constraint: their reasoning must be organized element-by-element against legally defined claim language, not simply against a mental model of the codebase. The failure mode is therefore not just lost context but legally indefensible analysis. Second, Lupu et al. [5] identified relevance assessment as an open problem in patent retrieval, but framed it primarily as a ranking and interface problem; our findings show that the deeper issue is the absence of integrated support for the interpretive work that follows retrieval—claim mapping, synonym management, and filing-date verification—which existing tools do not address at all. Third, Storey et al. [6] observed that developers alternate among cognitive strategies and argued that tools should accommodate this flexibility; we find a similar pattern among examiners, but with the added constraint that the output of their sensemaking must conform to a rigid legal structure, limiting how flexibly they can reorganize their analysis. Finally, while Pirolli and Card's sensemaking model [2] describes foraging and sensemaking as iteratively coupled, our participants described AI-assisted search tools that treat these as separate, one-shot operations (Need 5), suggesting that current tool design actively works against the iterative workflow that prior theory predicts experts will use.

These differences suggest that future research should more explicitly account for domains where information work is not only interpretive but also constrained. Our findings also suggest that future tool design could prioritize integration over automation. Rather than replacing examiner judgment with AI-driven search, the most impactful opportunities may lie in reducing manual overhead for tasks examiners already perform, such as synonym management, date filtering, and claim-to-evidence mapping.

5 DESIGN OF THE TOOL

5.1 Overview

What the tool is. Our system is a programmatic patent document analysis environment in which patent documents are treated as structured, queryable data rather than opaque full-text blobs. An examiner begins by loading one or more patent documents in PDF or plain-text form. The system parses each source into a `Document` → `Section` → `Passage` hierarchy, extracts patent metadata, and stores the result in a structured document representation that preserves section type, numbered paragraphs, claim anchors, and neighboring context. The examiner then selects the subset of documents they want to search over and writes a query in a lightweight textual DSL. The system executes the query at the passage level and returns matched passages with provenance: document title and identifier, section type and heading, passage index, optional paragraph or claim anchor, contextual neighbors, and a human-readable reason string explaining why the result matched.

The interface has two connected surfaces. The main search page combines a document manager, a DSL editor, and a scrollable list of result cards. Each result card supports lightweight downstream actions: the examiner can copy a citation trace directly from the card or send the passage into a separate claim-chart workspace. The second surface is a grouped claim-chart page in which evidence rows are organized under claim elements, edited, and exported as TSV or DOCX. This second surface is still prototype-grade, but it is included intentionally because our study showed that the handoff from search to claim mapping is itself one of the largest gaps in current examiner tooling. Figure 1 shows the current search interface, and Figure 2 shows the separate claim-chart workspace.

The search interface supports both explicit submission and live refinement. On a fresh page load, the system does not immediately run a broad query across all selected documents; this avoids hydrating unnecessary documents before the examiner has begun interacting. Once the examiner edits the query, changes the document selection, or presses the run button, the interface enables a 600ms debounced live-refresh loop so subsequent refinements update the result set in near real time. In parallel, the frontend opportunistically preloads the currently selected documents so the first query on that selected set need not hydrate each document one by one.

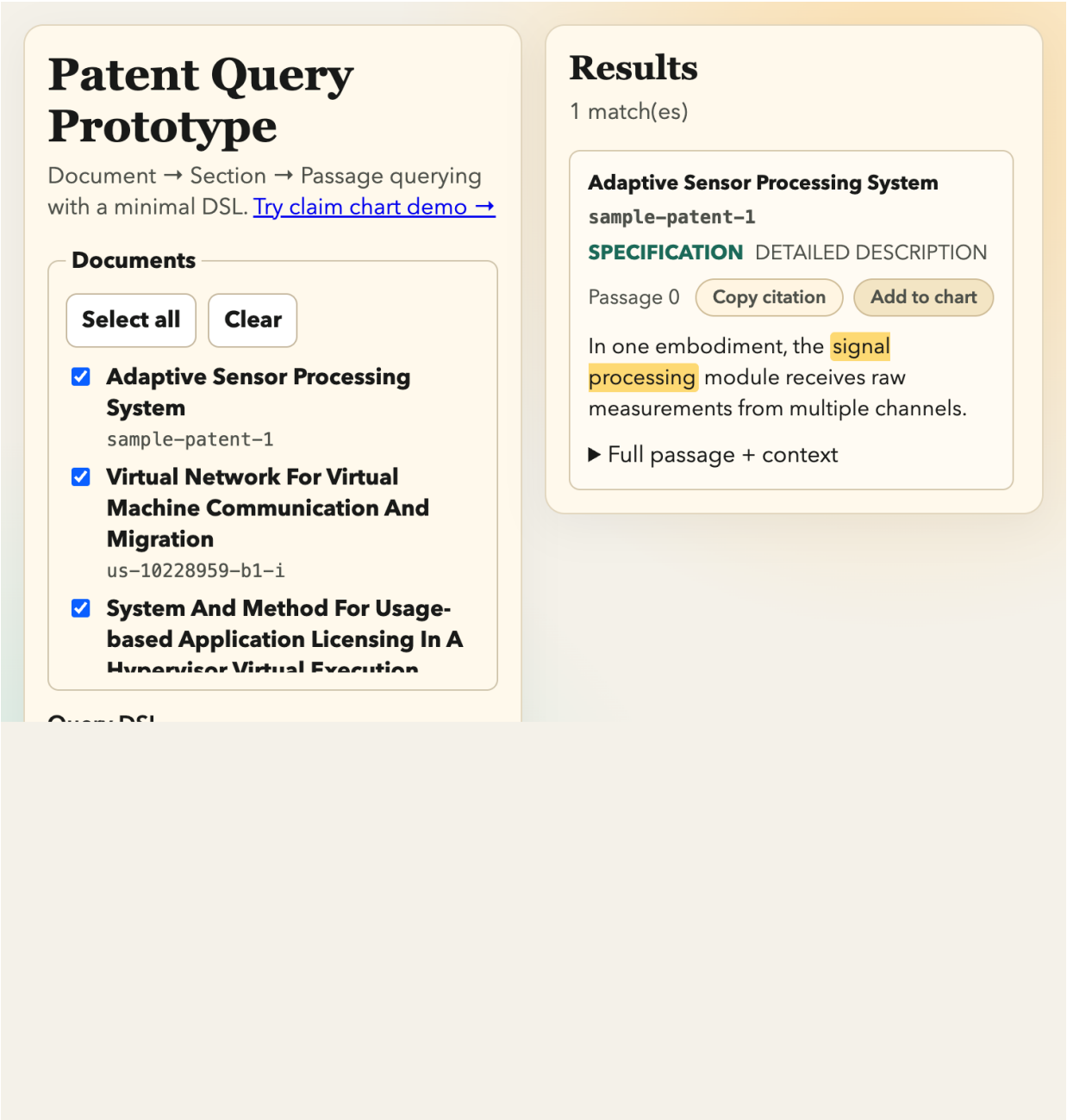

Figure 1: Main search interface. The left side exposes document selection and a DSL query editor; the right side shows passage-level results with section labels, contextual evidence, and lightweight actions for copying citations or sending passages into the chart workspace.

Why this tool, for this population. Every design decision in this section is traceable to a specific observation from our contextual inquiry sessions. The document model reflects the legal structure of patents as participants described it. The DSL operators map directly to the reasoning strategies participants actually used. The claim-chart handoff reflects the fact that retrieval and synthesis were never truly separate stages in our observations. The live refresh matches the iterative appraise-and-refine cycle that P02 articulated as the correct mental model for patent search. We describe these connections in detail below.

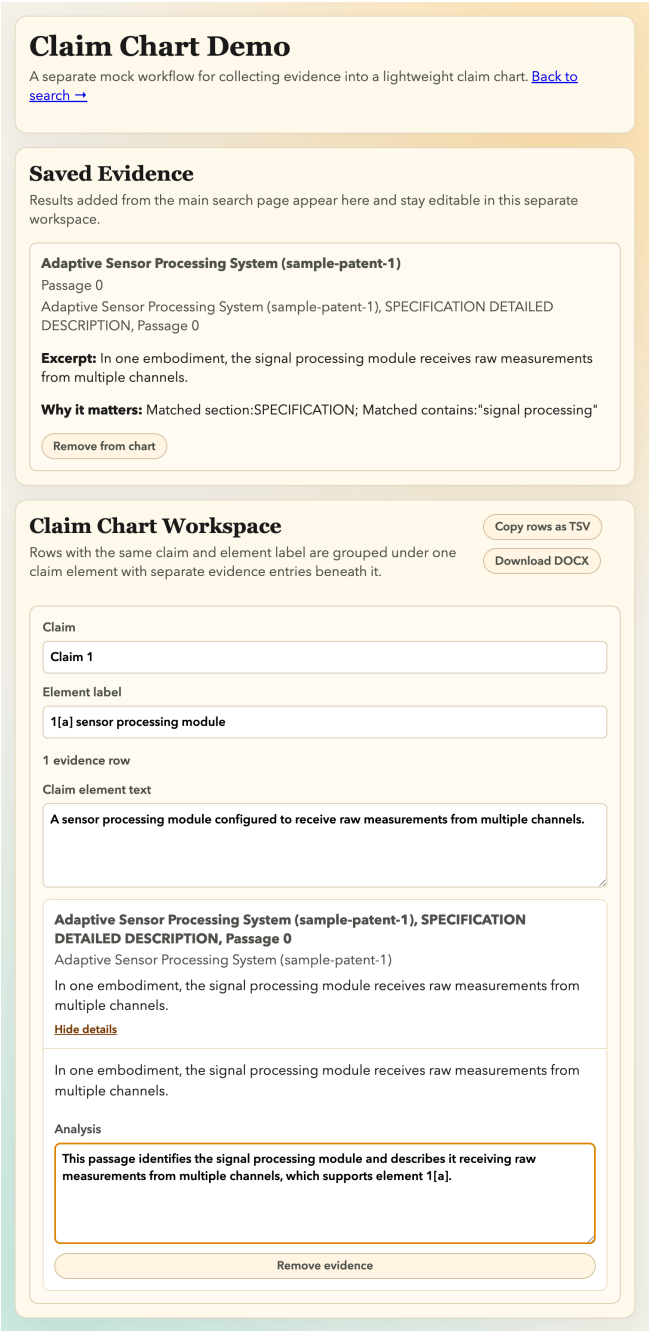

Figure 2: Claim-chart workspace. Evidence imported from search is grouped under a claim and element label, remains editable, and can be exported in lightweight structured formats such as TSV and DOCX.

5.2 Walkthrough: P03's Claim-Mapping Task

To make the design concrete, we walk through how our tool supports a task we observed during session P03. P03 was working a software patent application whose independent claim recited a system for *dynamic routing table updates*. After identifying several candidate references through classification-based search, P03 had to determine whether any of them disclosed the claimed routing mechanism, and if so, where the strongest supporting passages appeared. The process we observed involved repeated document opening, repeated `Ctrl+F` searches, and repeated manual note taking. P03 searched for one candidate term at a time, scanned every match regardless of section, recorded paragraph numbers manually when something looked useful, and then repeated the same cycle

for each synonym that seemed plausible (e.g., *routing table*, *forwarding table*, *FIB*, *routing cache*). P03 also had to manually inspect filing dates to determine which references were legally admissible.

With our tool, the same task proceeds as a single connected workflow.

Step 1: Load and scope the corpus. The examiner loads the seven candidate PDFs. The document manager lists them with their parsed titles and identifiers, while the backend indexes lightweight metadata immediately and defers full passage hydration until the documents are actually needed. To immediately filter to a legally admissible subset, the examiner can write:

```
meta.filingDate:<2018-03-15 AND cpc:"H04L␣45/00"
```

Listing 1: Filter by filing date and CPC before opening any document.

This eliminates references that postdate the application's effective filing date without the examiner having to open each reference in a separate viewer. This directly addresses Need 2: P03 described having to manually inspect each candidate result to determine which had a usable date.

Step 2: Query across the corpus for the claim element. The examiner then writes a query targeting the specific claim language. Rather than typing one term, inspecting results, and then starting over with a synonym, the examiner composes the full synonym set in a single query that executes across all selected documents simultaneously:

```
section:SPECIFICATION AND
  (contains:"routing␣table" OR contains:"
      forwarding␣table"
   OR contains:"FIB" OR contains:"routing␣cache")
```

Listing 2: Synonym expansion across a loaded corpus (Need 1).

The result set now contains only passages from the detailed specification—the section most likely to contain the technical teaching that maps to a claim element—that mention any of the four synonymous terms. Passages from the abstract, claims, and unrelated sections are excluded. Passages from all selected documents appear in one scrollable list. This addresses both Need 1 (synonym management) and Need 4 (section-scoped keyword localization) simultaneously.

Step 3: Assess results in context. Each result card displays the matched passage text alongside the passage immediately before and after it. P03 observed that keyword hits alone were often not sufficient to determine relevance—"you have to go through the whole patent" to understand whether a term was being used in the right sense. The neighboring passages eliminate most of that reading burden. Each card also exposes the paragraph or claim anchor, a `Copy citation` action, and an `Add to chart` action that sends the passage directly into the claim-chart workspace. This addresses the citation overhead P01 described while also reducing the manual handoff burden associated with Need 3. Figure 1 shows this result-card workflow in the current prototype.

Step 4: Collect and organize evidence. When the examiner identifies a promising passage, they add it to the claim-chart workspace. That page groups evidence rows by claim and element label, so multiple supporting passages for the same limitation can be managed together. The examiner can fill in the claim element text, add analysis, and export the grouped chart as TSV or DOCX. This is still a lightweight prototype rather than a full office-action authoring system, but it directly addresses the fact that participants currently bridge this stage with Word, paper, or personal templates. Figure 2 shows the current grouped workspace after evidence has been transferred from search.

Step 5: Iterative refinement. If the initial results contain false positives (e.g., passages about static routing that happen to mention "routing table"), the examiner narrows the query by adding a term without restarting:

```
section:SPECIFICATION AND
  (contains:"routing␣table" OR contains:"
      forwarding␣table")
  AND contains:"dynamic"
```

Listing 3: Narrowing by adding a required term.

Once the examiner has begun interacting, the result set updates within 600ms. The examiner can see immediately whether the added constraint reduced the result set to something manageable or eliminated too many relevant passages, and can adjust accordingly. This tight iteration loop was absent from every tool P03, P05, and P09 described. P02 articulated the correct model: effective search "really involves appraising the art as you search for it, and refining your search based on that" (Need 5). Our live refinement loop is the direct implementation of that observation.

5.3 The Document Abstraction

The hierarchy. Each patent document is parsed into a three-level hierarchy: `Document` → `Section` → `Passage`. A `Document` carries all extracted metadata: title, document identifier, filing and publication dates, application number, inventors, assignee, CPC classification codes, and domestic priority chain. A `Section` has a canonical type drawn from the set {TITLE, ABSTRACT, BACKGROUND, SUMMARY, SPECIFICATION, CLAIMS, OTHER} and a human-readable heading string extracted from the document. A `Passage` is an atomic unit of text within a section: a blank-line-separated paragraph, a USPTO-numbered paragraph (`[0042] ...`), or an individual claim.

This hierarchy was not chosen arbitrarily. During coding we noted that participants never described their information needs in terms of the full document; they consistently described them at the level of *sections* ("I want hits in the spec, not the abstract") and *passages* ("I need to know which paragraph to cite"). The three-level model is a direct structural translation of how patent examiners reason about documents.

Why section types are first-class. Patent sections are not stylistic divisions; they carry distinct legal functions that every participant in our study treated as categorical. The ABSTRACT is explicitly excluded as prior art under 35 U.S.C. §102(b)(2)(B) in certain conditions; the BACKGROUND describes the problem the invention addresses; the SPECIFICATION (detailed description) is the primary locus of technical teaching and is what examiners cite when writing a rejection; the CLAIMS define the legally protected scope. P05 was explicit: they wanted hits "in the background and description sections, not in the abstract or claims." No current tool surfaces

results at this level of section specificity. Our model makes section type queryable as a first-class predicate.

Passage-level anchors. Each `Passage` optionally carries one of three anchor types. A `paragraphId` records the USPTO paragraph number (e.g., `0042`) for numbered paragraphs in post-2001 publications. A `claimNo` records the claim number for passages parsed from the claims section. A `figureRefs` list records any figure references that appear within the passage text. The `paragraphId` directly addresses the citation overhead P01 described: when writing an office action, examiners must record exactly where in a reference the evidence appears. For published applications, this means paragraph numbers; for granted patents, column and line numbers. P01 called this process manual and frustrating. By extracting and storing the anchor at parse time, our system can surface it on the result card without requiring the examiner to open the document.

Metadata sidecars and selective hydration. In the current prototype, the document abstraction also shapes the storage and loading strategy. We separate lightweight document metadata from the full passage payload on disk so that startup can index the document set without materializing every passage body in memory. The backend can therefore list and filter loaded documents using metadata alone, while full document hydration is deferred until a document is actually queried or explicitly preloaded. This is not just an implementation detail: it reflects the same design principle as the frontend interaction model. Broad corpus knowledge should be cheap to keep available; expensive document structure should be loaded only when it supports an actual examiner action.

5.4 The Query DSL

The query language is a minimal composable DSL. The full grammar supports phrase containment, section type, paragraph anchor, CPC code, arbitrary metadata fields, and Boolean composition (`AND`, `OR`, `NOT`) with parenthesized sub-expressions and standard Boolean precedence. Every predicate in the language was motivated by a specific observed need; we describe each with its motivation below.

Content filtering (`contains:`). The simplest query is a phrase containment check over all passages in all selected documents:

```
contains:"prior art"
```

Listing 4: Basic phrase search.

This is the direct analog of a keyword search in any existing tool, but with a critical difference: the result is a list of individual passages, not a ranked list of documents. The examiner sees where in the document the term appears without having to open anything.

Section filtering (`section:`). Adding a `section:` predicate restricts results to passages from a specific document section. This was the single most consistently requested capability across our study:

```
section:SPECIFICATION AND contains:"virtual
    machine migration"
```

Listing 5: Section-scoped containment query (Need 4).

The predicate accepts any canonical section type and is case-insensitive. P05 wanted hits "in the background and description sections, not in the abstract or claims," and described the absence of this feature as something that "makes no sense for what I actually need to find." This predicate is the direct implementation of that need.

Synonym expansion (`OR`). Need 1 established that examiners must manually enumerate synonyms and run separate queries for each variant. The `OR` operator allows the full synonym set to be expressed in a single query:

```
section:CLAIMS AND
  (contains:"virtual machine" OR contains:"
      hypervisor"
   OR contains:"guest operating system")
```

Listing 6: OR for synonym expansion across a corpus (Need 1).

This returns all claim passages from all selected documents that mention any of the three terms. P04 described maintaining a shared synonym crosswalk; this query structure makes that crosswalk executable rather than just a reference document, and applies it across a corpus in one step.

Negation (`NOT`). The `NOT` operator excludes passages from the result set. A practical use case is filtering the introductory boilerplate paragraph that many specifications begin with, which often contains broad terminology that matches almost any query:

```
section:SPECIFICATION AND contains:"network
    interface"
  AND NOT paragraph:1
```

Listing 7: NOT to exclude boilerplate preamble.

Paragraph anchoring (`paragraph:`). The `paragraph:` predicate retrieves a specific numbered passage by its USPTO paragraph anchor. This is useful when an examiner returns to a document in a subsequent session and needs to pull up a known-relevant paragraph without re-reading the specification:

```
section:SPECIFICATION AND paragraph:42
```

Listing 8: Pinpoint retrieval by paragraph anchor (Need 4).

This also enables cross-document comparison: an examiner who has noted paragraph `[0042]` as relevant in one reference can query the same anchor across other loaded documents to see whether they share structurally similar placement for the same concept.

Metadata filtering (`meta.` and `cpc:`). CPC classification codes and arbitrary metadata fields narrow the document scope before content is examined. This reflects the corpus-scoping workflow that multiple participants described: they first constrain the set of candidate documents by classification, then search their content.

```
cpc:"H04L 45/00" AND section:SPECIFICATION
  AND contains:"link-state advertisement"
```

Listing 9: CPC scope then section filter.

```
meta.filingDate:<2019-06-01
  AND section:CLAIMS AND contains:"neural
      interface"
```

Listing 10: Date constraint to enforce admissibility (Need 2).

The date predicate directly addresses Need 2. P05 described completing an entire office action on a reference that postdated the application's effective filing date, requiring a complete redo. P06 called the absence of reliable date filtering "definitely a very good fix that can be done."

The current prototype also supports metadata conveniences that reduce the gap between raw metadata structure and the way examiners naturally formulate queries. In addition to nested metadata paths such as `meta.assignee.name`, the implementation supports direct aliases such as `meta.pubDate`, `meta.appNo`, `meta.assigneeName`, and `meta.inventorName`. It also supports substring and prefix operators for metadata exploration as well as derived helper fields such as `meta.priorityDate`, `meta.effectiveDate`, and `meta.admissibilityDate`. These helpers do not solve the legal admissibility problem completely, but they move date reasoning closer to the query layer than any tool participants described using.

Full composability. All predicates compose with arbitrary nesting. The following query reflects the kind of compound search P03 assembled across multiple tool sessions during our study—expressed here in a single executable string that can be saved, shared, and rerun:

```
meta.filingDate:<2018-03-15 AND section:
    SPECIFICATION AND
  (contains:"routing␣table" OR contains:"
      forwarding␣table"
   OR contains:"FIB")
  AND contains:"dynamic"
```

Listing 11: Compound query for a single claim element.

Each result card shows the matched passage text, the section type and heading, the paragraph or claim anchor if present, the immediately preceding and following passages for context, and a match-reason string that names each predicate that matched (e.g., `section:SPECIFICATION`, `contains:"routing table"`). The examiner can use this string directly as a citation trace in a claim chart, addressing the absence of integrated claim-mapping support identified in Need 3.

5.5 Key Design Decisions

Decision 1: Passage granularity with neighboring context. The most fundamental question in designing a retrieval tool is the unit of retrieval. Existing tools return documents; sentence-level tools exist in academic NLP but are unfamiliar to practitioners. We chose the passage because it corresponds to how patent documents are legally structured and how examiners described their information needs: they wanted "where in the patent" a term appeared, not just "which patent."

However, our analysis sessions surfaced a secondary problem: a passage clipped from its context can be misleading or uninterpretable. P03 was explicit that "you have to go through the whole patent" in part because the keyword match by itself does not reveal whether the surrounding text confirms or contradicts the relevance judgment. We resolved this by including the immediately preceding and following passages in every result card. This adds enough local context for a relevance judgment without requiring the examiner to load the full document. The tradeoff—longer result cards, more scrolling—was acceptable because it directly replaces the longer task of opening each document to read around each hit.

Decision 1b: Search and synthesis remain connected. An early version of the prototype stopped at passage retrieval. That would have been enough to demonstrate the DSL and the passage-level model, but it would have ignored one of the strongest findings from our study: the handoff from retrieval to claim mapping is itself a major source of friction. We therefore chose to expose lightweight downstream actions directly on result cards and to build a separate claim-chart workspace rather than treating search as the end of the interaction. This does not mean the current claim-chart feature set is complete. It means that our design explicitly recognizes claim mapping as part of the same workflow rather than a separate phase left entirely to external tools.

Decision 2: Section type as a predicate with a closed vocabulary, not a metadata field. An early design used a generic `meta.` namespace for all document properties, which would have encoded section type as `meta.sectionType:"SPECIFICATION"`. We changed this to a dedicated `section:` predicate with a closed vocabulary after analyzing how participants talked about sections during the CI sessions.

Participants never referred to sections by arbitrary heading text; they referred to them by legal role. P05 said "the background and description sections," not "the section whose heading contains 'Detailed Description.'" This suggested that the model needed a closed, normalized vocabulary of legal section types rather than a string-match against headings—particularly because heading text varies across issuers and publication formats (e.g., "Detailed Description of the Preferred Embodiments" vs. "Detailed Description"). Normalizing to canonical types at parse time and exposing them through a dedicated predicate syntax reflects the conceptual model participants actually held.

Decision 3: Boolean composition over ranked retrieval. We made an early decision to build a Boolean filter model rather than a ranked retrieval model (e.g., TF-IDF, BM25, or embedding similarity). This was one of the most consequential choices in the design and was almost entirely driven by Need 5.

P02 described effective search as "appraising the art as you search for it, and refining your search based on that." This articulates a specific epistemic structure: the examiner's understanding of what they are looking for evolves as they look. Ranked retrieval systems assume the query is fixed and the model's job is to order the corpus against it; they do not expose *why* a document ranked where it did, making it opaque which dimension to refine when results are wrong. P09 described the USPTO similarity search as producing 300 references "of which 2 or 3 are actually useful," with no clear signal about what to adjust. P06 identified the core problem: the model has no awareness of domain context, so a search for "user interface" in a medical device application returns banking software results.

A Boolean DSL is transparent by design. Every predicate either matches or does not, and the match-reason string on each result card names exactly which predicates fired. When a result is missing, the examiner can add an `OR` clause; when there are too many false positives, they can add an `AND` constraint. Each edit is a one-token change with an immediately visible effect. This transparency is

precisely what makes it compatible with the iterative appraise-and-refine strategy that participants described as their actual method.

Decision 4: Textual DSL over a GUI filter panel. We considered implementing the same predicate semantics as a graphical filter panel with dropdowns, checkboxes, and date pickers. The textual DSL won out for reasons that came directly from the CI sessions.

First, patent examiners already write Boolean query strings as a core job skill. P04, P07, and P10 each described constructing Boolean queries as standard practice. A textual DSL is a familiar interaction mode for this population; a filter panel would introduce a new interaction paradigm for examiners who already think in terms of query strings.

Second, and more importantly, a textual query is a first-class persistent artifact in a way that a GUI selection state is not. P01 maintained a Word table for claim mapping; P03 used inline bullet points; P05 copied claims into a Word document. All of these are workarounds for the absence of integrated documentation support (Need 3). A query string can be pasted directly into a claim chart as a citation trace: "found via `section:SPECIFICATION AND contains:"routing table"`, paragraph [0042]." A GUI filter panel cannot be recorded that way. The DSL makes queries *shareable*—with colleagues, across cases, and over time—in a way that a stateful UI control does not.

Third, the live debounce refresh means that typing in the text box already behaves like a responsive filter panel: each character edit produces updated results within 600ms. The perceived interactivity difference between a textual DSL and a GUI panel is therefore small, while the artifact value of the textual form is substantially higher.

Decision 5: Preprocess broad, hydrate narrow. As the prototype expanded from a small demo to a multi-document tool, a new design question emerged: how much of the corpus should be processed eagerly and how much should remain deferred until the user acts? A naive design would hydrate every parsed document into memory on startup. That keeps the first query fast, but it also wastes work on documents the examiner may never query. At the other extreme, a purely on-demand design makes every first query pay a document-loading cost even when the tool already has strong signals about what the examiner is likely to search next.

We resolved this tension by separating preprocessing into two layers. The system eagerly indexes lightweight metadata for all parsed documents so the UI can present and filter a corpus cheaply. It then lazily hydrates full documents on first access, while also opportunistically preloading the currently selected document set after explicit user interaction. This design fits the workflow we observed: examiners often know the subset of candidate references they are actively working with, but they do not want the system to waste effort on every possible document in the corpus. The resulting strategy is simple to describe: preprocess broad, hydrate narrow.

6 TECHNICAL CHALLENGES AND TECHNICAL SOLUTIONS

6.1 Challenge 1: Parsing Heterogeneous Patent Formats into a Uniform Hierarchy

6.2 Challenge 2: Designing a Composable Filter Model for the DSL

6.3 Challenge 3: Provenance and Context Reconstruction

6.4 Remaining Work

7 IMPLEMENTATION

The backend is implemented in Python 3 using FastAPI and exposes three endpoints: `/documents` (list loaded documents), `/documents/parse` (trigger parsing of raw files), and `/query` (execute a DSL query). The frontend is a thin React/TypeScript client built with Vite. The parser pipeline uses `pypdf` for text extraction from PDFs and a custom recursive-descent parser for the DSL. Structured JSON documents are stored in `backend/data/parsed/`. The full implementation is available at https://github.com/shanayamalik/cs294-sp26.

8 DISCUSSION

9 ADDITIONAL NOTES

ACKNOWLEDGMENTS

We thank Professor Sarah E. Chasins for her guidance throughout this project and the CS 294 course at UC Berkeley. We also thank the ten patent examiners who generously shared their time, expertise, and workflows with us.

REFERENCES

[1] A. J. Ko, B. A. Myers, M. J. Coblenz, and H. H. Aung. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Trans. Softw. Eng.*, 32(12):971–987, 2006.
[2] P. Pirolli and S. Card. The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis. In *Proc. Int. Conf. Intelligence Analysis*, volume 5. MITRE, McLean, VA, 2005.
[3] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli. Modern Code Review: A Case Study at Google. In *Proc. ICSE-SEIP '18*, pages 181–190. ACM, 2018.
[4] D. M. Russell, M. J. Stefik, P. Pirolli, and S. K. Card. The Cost Structure of Sensemaking. In *Proc. CHI '93*, pages 269–276. ACM, 1993.
[5] M. Lupu, K. Mayer, N. Kando, and A. J. Trippe. *Current Challenges in Patent Information Retrieval*, volume 29 of *The Information Retrieval Series*. Springer, 2013.
[6] M.-A. D. Storey, F. D. Fracchia, and H. A. Müller. Cognitive Design Elements to Support the Construction of a Mental Model during Software Exploration. *J. Systems and Software*, 44(3):171–185, 1999.
[7] N. J. Nersessian. *Creating Scientific Concepts*. MIT Press, 2008.
[8] H. Beyer and K. Holtzblatt. *Contextual Design: Defining Customer-Centered Systems*. Morgan Kaufmann, 1997.
[9] J. Lubin and S. E. Chasins. How Statically-Typed Functional Programmers Write Code. *Proc. ACM Program. Lang.*, 5(OOPSLA):Article 155, 2021.
[10] T. Zhang, B. Hartmann, M. Kim, and E. Paez. Enabling Data-Driven API Design with Community Usage Data: A Need-Finding Study. In *Proc. CHI '20*, pages 1–13. ACM, 2020.
[11] V. Braun and V. Clarke. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology*, 3(2):77–101, 2006.